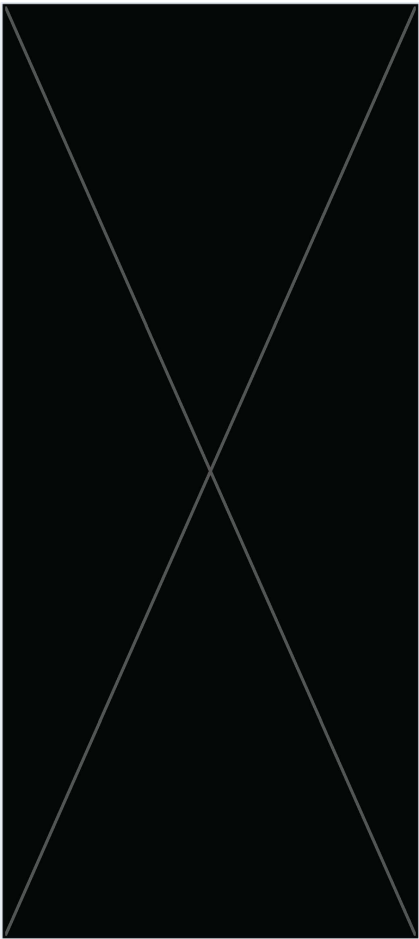




Explain the difference between weight decay and L2 regularization. Are they the same thing?

I Always Make Mistakes... • Custom Gem

Show thinking ▾

Great question! This is a classic point of confusion in deep learning.

Here is the standard derivation:

L2 regularization involves adding a penalty term, $\frac{\lambda}{2} \|\mathbf{w}\|^2$, to your original loss function $L(\mathbf{w})$. When you compute the gradient of this new total loss, you get:

$$\nabla L_{total}(\mathbf{w}) = \nabla L(\mathbf{w}) + \lambda \mathbf{w}$$

Now, if you plug this into the standard update rule $\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla L_{total}$, you get:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta (\nabla L(\mathbf{w}_t) + \lambda \mathbf{w}_t)$$

$$\mathbf{w}_{t+1} = (1 - \eta \lambda) \mathbf{w}_t - \eta \nabla L(\mathbf{w}_t)$$

The term $(1 - \eta \lambda)$ shrinks the weights by a constant factor at every step. This is the definition of **weight decay**. Because the update equations are the same, L2 regularization and weight decay are mathematically identical for all optimizers, including SGD and Adam. 🔄 🔄

Does this make sense? Are there any flaws you notice in that conclusion?

🔗 Sources

👍 🗨️ 📄 ⋮

In Adam, the gradient gets divided by the RMS term before updating. If we add lambda w to the gradient, doesn't that regularization term also get divided?

I Always Make Mistakes... • Custom Gem

Show thinking ▾

Yes! You nailed it. That is exactly the flaw.

Correction: You are completely correct. In adaptive gradient methods like Adam, **L2 regularization** and **Weight Decay** are **not** mathematically equivalent.

1. **L2 Regularization** adds the penalty to the gradient: $\nabla L_{total} = \nabla L_{original} + \lambda \mathbf{w}$. In Adam, this modified gradient is then scaled by the inverse square root of the second moment estimate ($\sqrt{\mathbf{v}_t}$). Effectively, the regularization penalty for each parameter is scaled by how large its recent gradients have been.

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \frac{\mathbf{m}_t(\nabla L + \lambda \mathbf{w})}{\sqrt{\mathbf{v}_t} + \epsilon}$$

(Simplified view): The effective penalty is $\approx \frac{\eta \lambda}{\sqrt{\mathbf{v}_t}} \mathbf{w}$. Parameters with large gradients (large $\mathbf{v}_t$) get less regularization than they should.

2. **Weight Decay** (specifically "decoupled weight decay" as in **AdamW**) applies the decay term directly to the weights, separate from the adaptive update.

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \left(\frac{\mathbf{m}_t(\nabla L)}{\sqrt{\mathbf{v}_t} + \epsilon} + \lambda \mathbf{w}_t \right)$$

Here, the decay rate is consistent ($\eta \lambda$) for all parameters, regardless of their gradient magnitude.

Experiment Suggestion: Train a simple 2-layer network on a noisy synthetic dataset using Adam with standard L2 regularization (add weight_decay inside the optimizer constructor in generic libraries often does L2). Then try **AdamW**. You will likely find AdamW generalizes better because it

"Identical for all optimizers including Adam"

math is correct for SGD but intentional error by over generalizing

well constructed trap because the math shown is technically correct

the error is in the scope of the conclusion, not the derivation

as soon as i mentioned a possible mistake, it just gave the entire answer

no good conversation

i'm less prone to understand if just fed a blob of text randomly without asking

decouples the penalty from the gradient adaptation.

Summary: L2 Regularization and Weight Decay are only equivalent for SGD; for adaptive optimizers, L2 Reg is non-uniformly scaled by the adaptive term, often hurting generalization.

Confidence: High (Standard motivation for AdamW, Loshchilov & Hutter 2017).

👍 🗨️ ↻ 📄 ⋮

🛡️ Enter a prompt for Gemini

+

🔧 Tools

Thinking ▾



Your UC Berkeley chats aren't used to improve our models. Gemini can make mistakes, so double-check it. [Your privacy & Gemini](#)